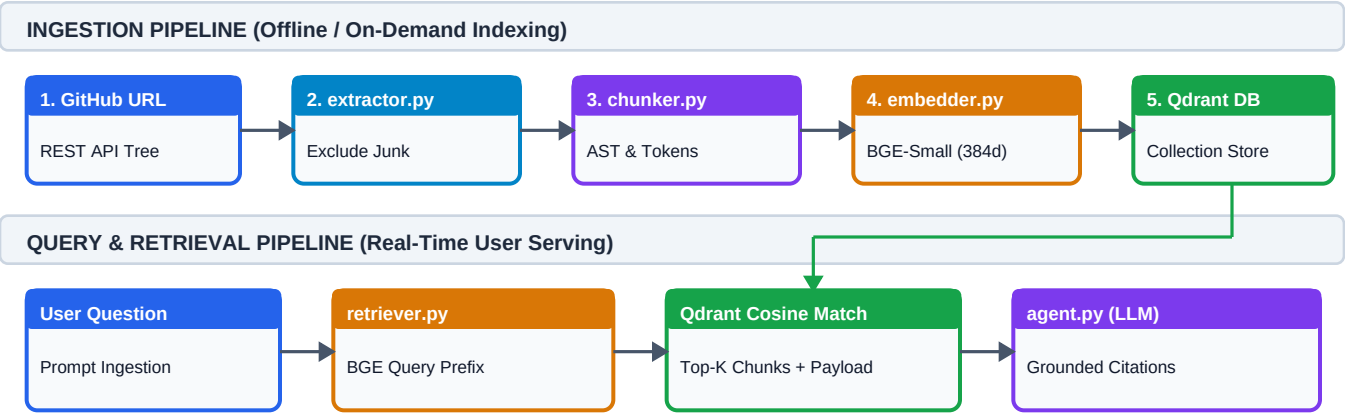


Gito — GitHub Repository AI Partner

Comprehensive Technical Architecture, Pipeline Flow & Developer Reference Guide

Executive Overview: Gito is an autonomous codebase agent built on a **Hybrid RAG (Retrieval-Augmented Generation)** architecture. It ingests any public or private GitHub repository via API, filters junk and binary files, splits code semantically by function (via AST) and documentation by token windows, embeds chunks using the state-of-the-art **BAAI/bge-small-en-v1.5** model, and indexes them into a localized **Qdrant** vector database. When a user queries the repo, Gito performs asymmetric vector similarity search and injects grounded context snippets into an LLM answering layer to deliver cited, accurate answers.

1. High-Level System Architecture & Flowchart



2. Core Technology Stack Cheat Sheet

Component	Technology / Library	Key Role & Implementation Detail
Repo Extraction	GitHub REST API v3 requests	Recursive tree fetching (git/trees?recursive=1). Downloads raw files while bypassing binaries, lock files, and node_modules.
Code Chunking	Python ast Regex Heuristics	Extracts discrete functions, methods, and classes. Preserves module-level headers (imports & global variables).
Doc Chunking	tiktoken (cl100k_base)	Hierarchical 3-tier splitting: Headings -> Paragraphs -> Sentences. Targets 1,000–1,500 token windows without orphan headings.
Vector Embeddings	BAAI/bge-small-en-v1.5 sentence-transformers	384-dimensional dense vectors. L2-normalized so cosine similarity is computed via dot product. Queries use asymmetric instruction prefix.
Vector Database	qdrant-client (v1.19+) Local Persistent Storage	On-disk storage in ./qdrant_db/ (no Docker required). Stores full chunk payload (filepath, name, type, code content) for fast lookup.
LLM Layer	groq / openai Pluggable Architecture	Grounded system prompt enforcing [file:function] citations. Supports real-time token streaming and dry-run fallback.

3. Deep-Dive: How Backend Features Were Built

A. Smart Ingestion & Exclusion Engine (extractor.py)

Standard repo clones bloat embeddings with multi-megabyte lock files, minified bundles, and compiled binaries. Gito avoids this completely by querying the recursive Git tree endpoint and evaluating every path against strict exclusion predicates:

- **Excluded Directories:** .git/, node_modules/, venv/, dist/, build/, __pycache__/, target/, vendor/
- **Excluded Binaries/Media:** Images (.png, .jpg), video (.mp4), archives (.zip, .tar.gz), binaries (.exe, .dll), datasets (.csv, .sqlite)
- **Excluded Locks & Configs:** package-lock.json, yarn.lock, .env, .gitignore, and linter dotfiles (.rc, .rc.json)
- **Direct Raw Streaming:** Downloads valid source files directly from raw.githubusercontent.com, preserving repository folder structure.

B. Dual-Strategy Chunking Engine (chunker.py)

RAG systems fail when code is split arbitrarily across line numbers. Gito implements two distinct, tailored chunking strategies:

STRATEGY A: CODE CHUNKING

- Target: .py, .js, .ts, .jsx, .tsx, .java, .go, .rs, .c, .cpp
- Python: AST (Abstract Syntax Tree) splits by Class/Function
- Polyglot: Regex pattern heuristics for definitions/exports
- Header Extraction: Imports & constants -> 'module_header'
- **Output: 1 chunk per logical function with exact name metadata**

STRATEGY B: DOC & README CHUNKING

- Target: .md, .mdx, .rst, .txt, README, LICENSE, GUIDES
- Token Window: 1,000 to 1,500 tokens (tiktoken cl100k_base)
- 3-Tier Split: Markdown Headings -> Paragraphs -> Sentences
- Anti-Orphan: Tiny heading fragments merged into adjacent chunk
- **Output: Dense contextual passages with token count payload**

C. BGE Embedding & Vector Normalization (embedder.py)

Gito utilizes **BAAI/bge-small-en-v1.5** (384 dimensions). This model achieves MTEB benchmark performance while running fast on standard CPUs. Key mechanics:

- **Asymmetric Query Prefixing:** BGE requires queries to be prepended with "Represent this sentence: " to bridge the semantic asymmetry between short questions and dense code blocks. Documents/chunks are embedded as-is.
- **L2-Normalization:** Vectors are normalized to unit length ($\|v\| = 1.0$). This mathematically reduces cosine similarity to a simple vector dot product, enabling lightning-fast distance computations in Qdrant.

D. Qdrant Local Multi-Tenant Storage (qdrant_store.py)

Instead of storing loose binary arrays on disk, Gito integrates **Qdrant** in embedded file mode (./qdrant_db/):

- **Sanitized Multi-Repo Collections:** Each repository receives its own isolated Qdrant collection named after owner_repo (e.g., octocat_hello-world).
- **Rich Payload Metadata:** Along with the 384-d vector, each point stores: filepath, chunk_type, name, content, and language. No secondary database lookups are needed.
- **Qdrant 1.19+ API:** Uses modern query_points() with optional payload filtering by chunk_type.

4. Code Map & Developer Reference (What Each File Does)

Use this cheat sheet whenever you need to modify, optimize, or extend any part of the Gito backend:

File	Core Classes & Functions	What It Does & When to Modify
extractor.py	<code>extract_repo()</code> <code>is_excluded()</code> <code>parse_github_url()</code>	Handles GitHub API repository download and filtering. Modify when: Adding new ignored file extensions, changing GitHub branch rules, or supporting enterprise GitHub URLs.
chunker.py	<code>chunk_repository()</code> <code>chunk_code_file()</code> <code>chunk_doc_file()</code> <code>_chunk_python()</code>	Splits files into semantic chunks. Modify when: Adding AST support for new programming languages, adjusting the 1,000–1,500 token window, or tuning markdown heading split rules.
embedder.py	<code>Embedder</code> <code>.embed_chunks()</code> <code>.embed_query()</code>	Generates 384-d BGE vector embeddings. Modify when: Changing the embedding model (e.g., to BGE-base or voyage-code), altering batch sizes, or modifying query prefixes.
qdrant_store.py	<code>QdrantVectorStore</code> <code>.index_chunks()</code> <code>.search()</code> <code>.list_collections()</code>	Manages Qdrant vector database storage & search. Modify when: Changing database location (./qdrant_db), adding remote cloud credentials, or adding payload filter fields.
retriever.py	<code>Retriever</code> <code>.retrieve()</code> <code>.format_context()</code>	Orchestrates query embedding and vector search matching. Modify when: Tuning top_k defaults, setting minimum similarity thresholds, or modifying the context block prompt template.
agent.py	<code>GitoAgent</code> <code>.answer()</code> <code>SYSTEM_PROMPT</code>	The LLM answering layer. Modify when: Editing Gito's system prompt instructions, adding new LLM providers (Anthropic, DeepSeek), or adjusting temperature and streaming parameters.
main.py	<code>main()</code> <code>chat_loop()</code> <code>index_new_repository()</code> <code>select_or_add_repository()</code>	Unified CLI endpoint connecting the complete pipeline. Modify when: Adding new CLI flags, enhancing menu options, or hooking up backend events to a frontend UI.

5. Step-by-Step Query & Answering Lifecycle

- **1. User Question Ingestion:** User types a query (e.g. "How does authentication work?").
- **2. Query Normalization & Embedding:** Query is prefixed with "Represent this sentence: ", passed to BGE-Small, and normalized to unit length.
- **3. Qdrant Cosine Retrieval:** Qdrant executes cosine distance comparison against all vectors in the collection, returning top-\$k\$ hits with payloads.
- **4. Prompt Construction & Grounding:** Retrieved code/doc chunks are formatted into numbered source blocks with metadata: [Source 1] auth.py (auth.py:verify_token) | Score: 0.8120.
- **5. LLM Synthesis & Streaming:** The LLM receives the system prompt + context and streams a cited, grounded response directly to the user.

CLI Quickstart Command: `python main.py` (Launches interactive menu to select existing repo or index a new one)